TLB Introduction and Design

Sparsh Mittal

TLB = Translation Lookaside Buffer

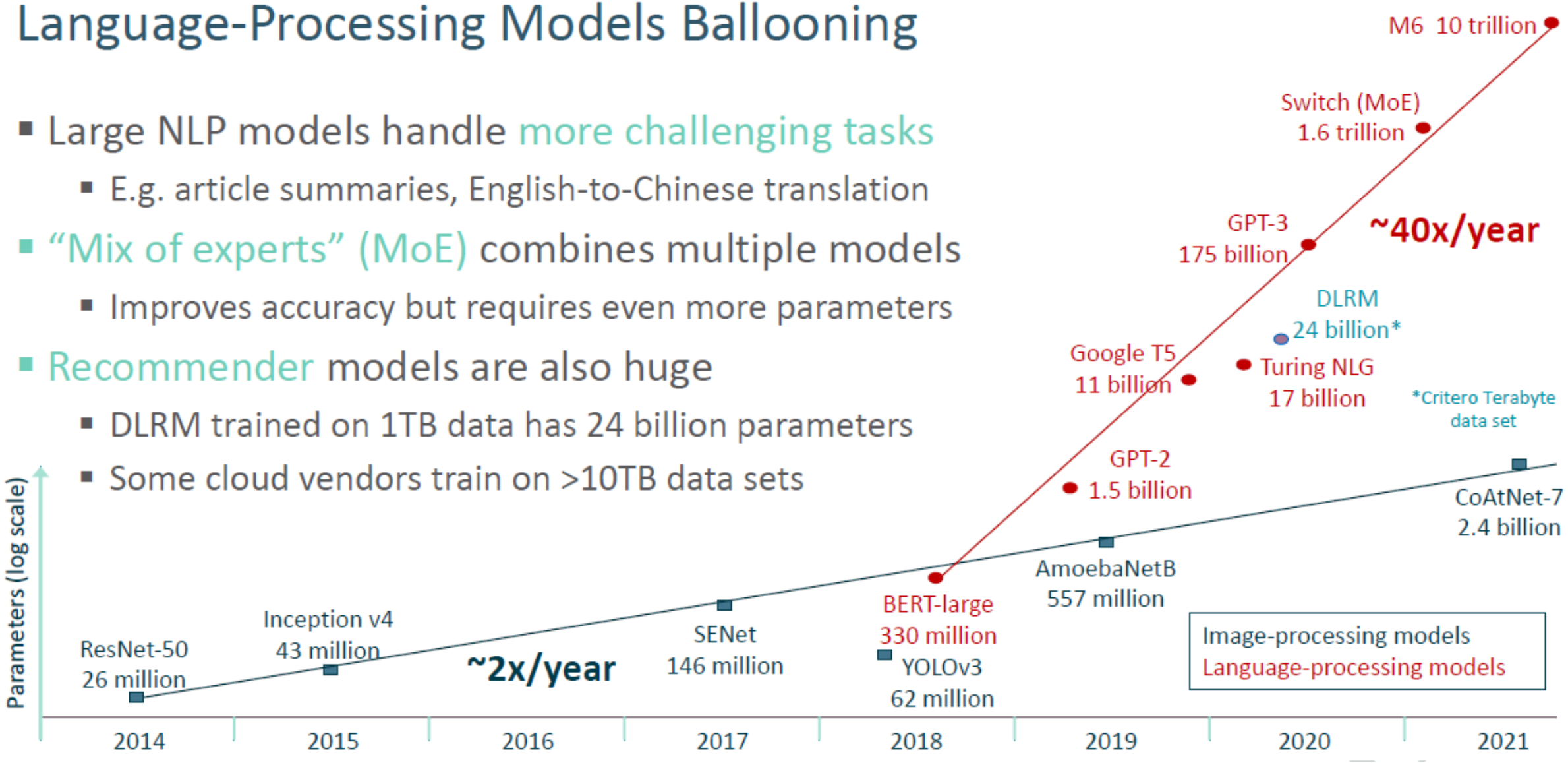
Courtesy for some slides: O. Mutlu and S. Khan

Acronyms

- VA/PA = virtual/physical address
- HW/SW = hardware/software
- ITLB/DTLB = instruction/data TLB
- PTE = page table entry
- Reg = register
- VM = virtual memory

Language-Processing Models Ballooning

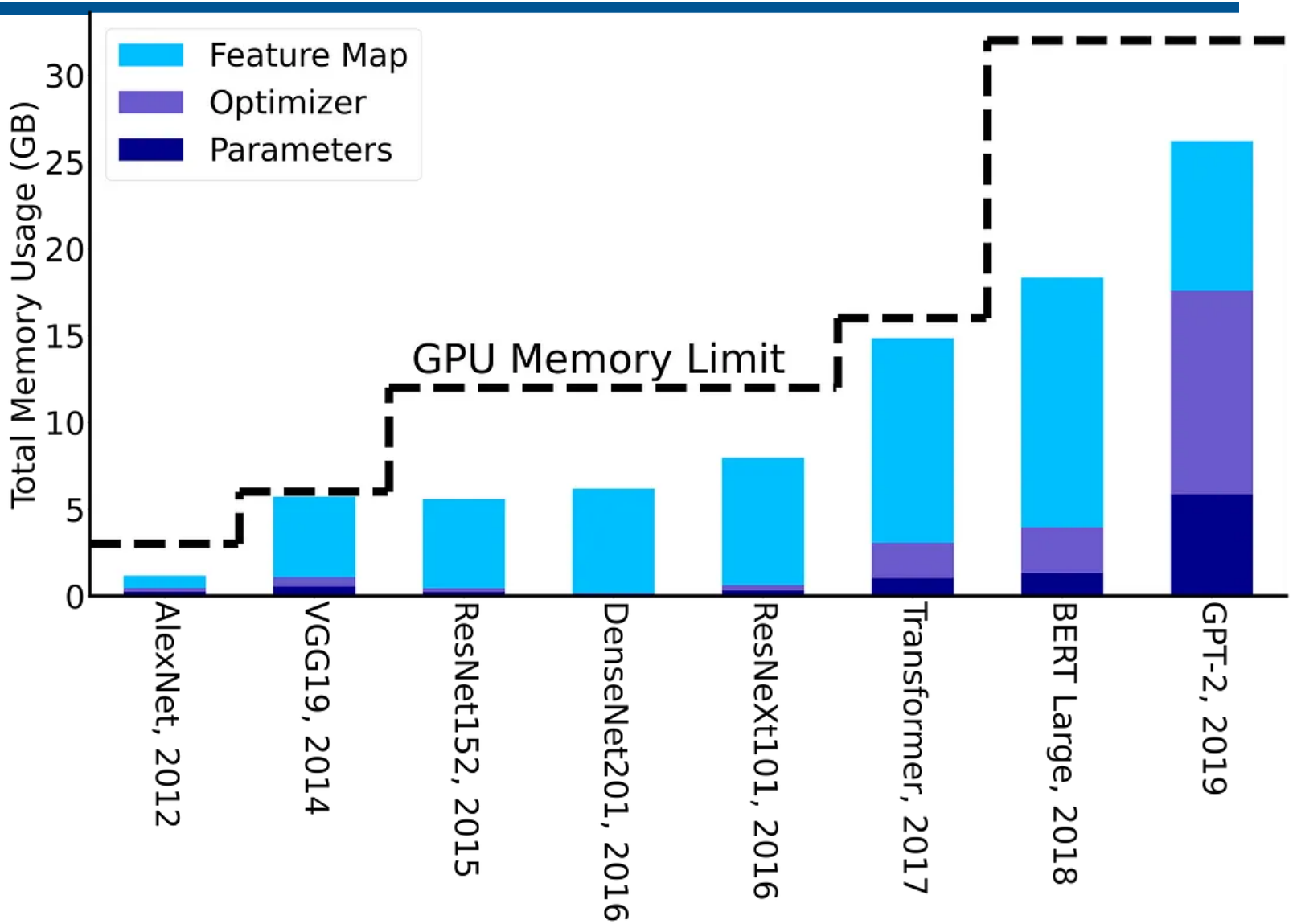
- Large NLP models handle more challenging tasks
  - E.g. article summaries, English-to-Chinese translation
- "Mix of experts" (MoE) combines multiple models
  - Improves accuracy but requires even more parameters
- Recommender models are also huge
  - DLRM trained on 1TB data has 24 billion parameters
  - Some cloud vendors train on >10TB data sets



Out of course

More memory ➔ more AI innovation

- The amount of memory required to train different NN models.
- Every time the GPU memory capacity is increased, scientists have designed newer models.
- ➔ Providing more memory could further allow new innovations.



Out of course

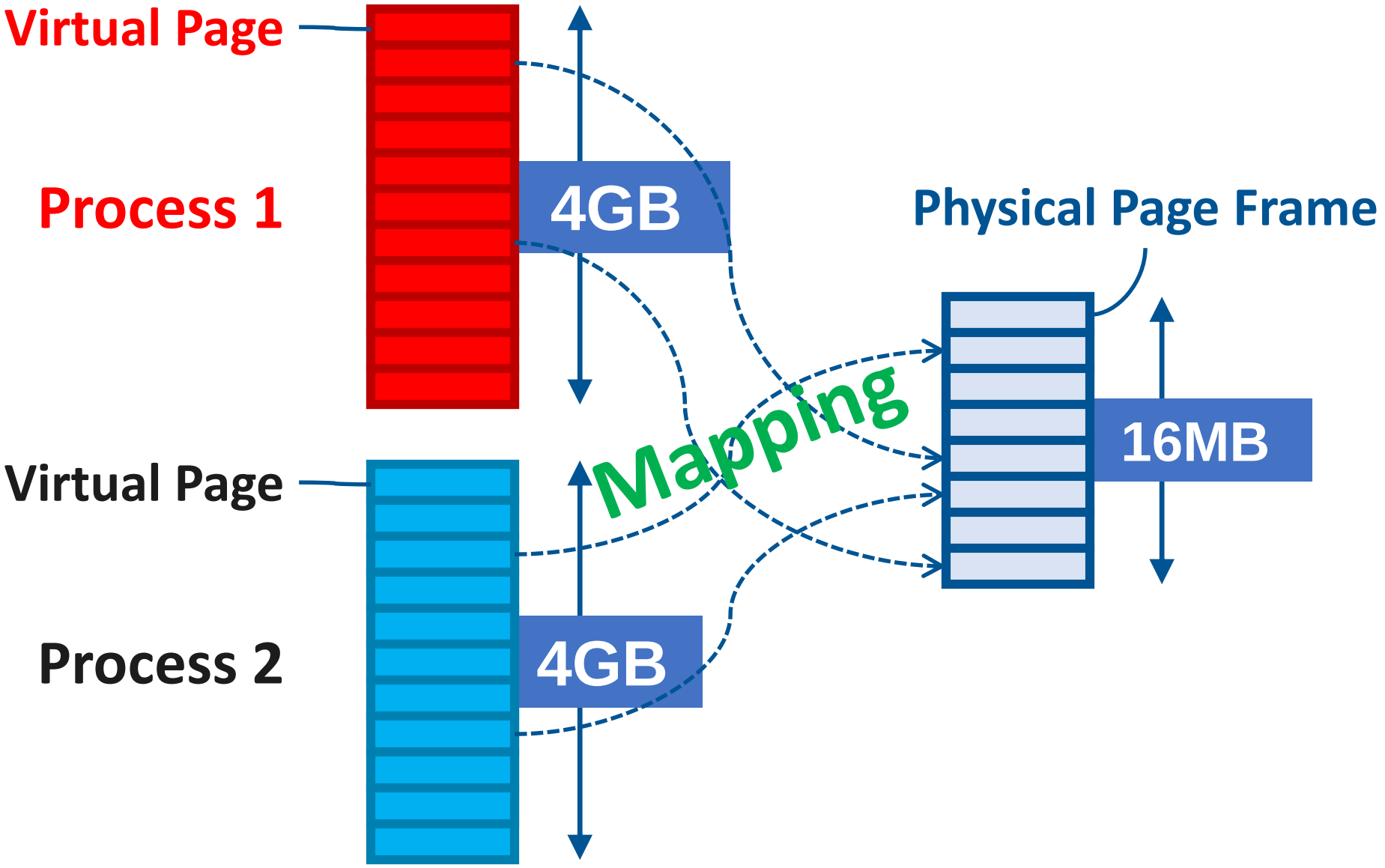
Virtual Memory

- Idea: Give the programmer the illusion of a large address space
  - So that he/she does not worry about running out of memory
- Programmer can assume he/she has “infinite” amount of physical memory
  - Really, it is the amount specified by the address space for a program
- Address generated by each instruction in a program is a “virtual address”
  - i.e., it is not the physical address used to address main memory
- An “address translation” mechanism maps this address to a “physical address”
  - Address translation mechanism can be implemented in hardware and software together

Virtual Pages, Physical Frames

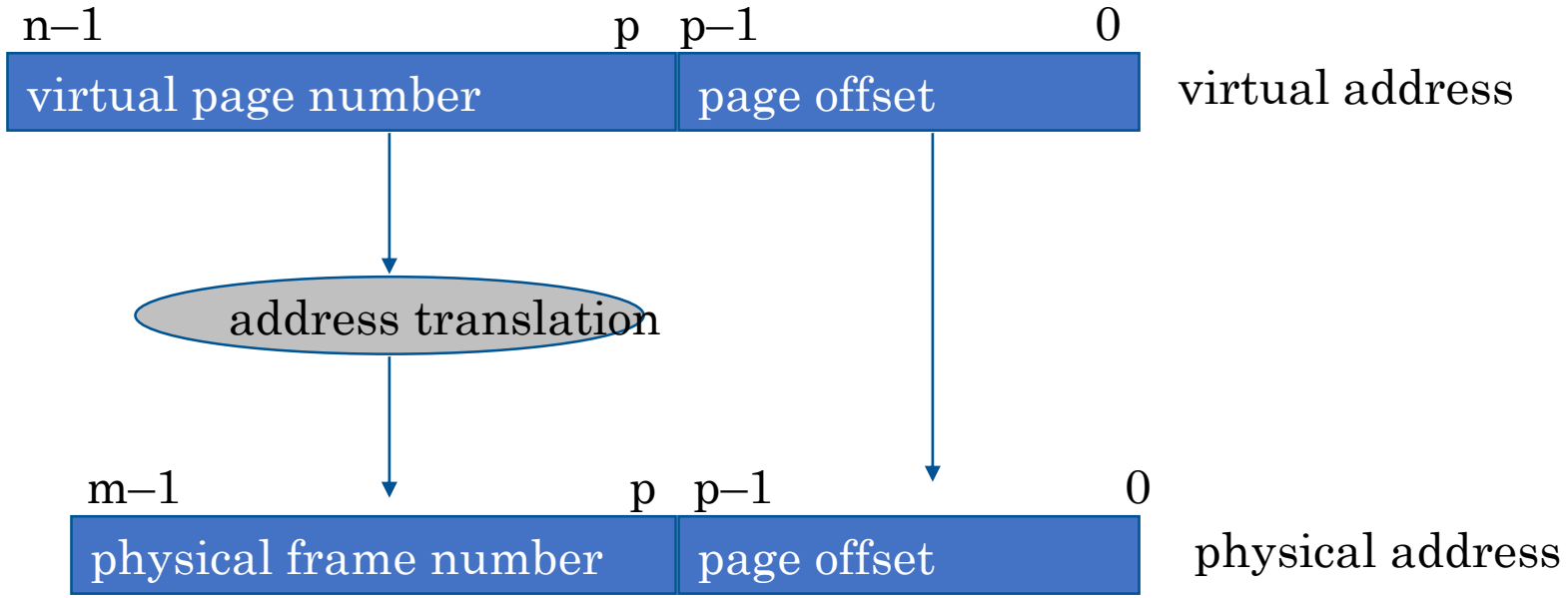
- Virtual address space divided into pages
- Physical address space divided into frames
- A virtual page is mapped to a physical frame
  - Assuming the page is in memory
- If an accessed virtual page is not in memory, but on disk
  - Virtual memory system brings the page into a physical frame and adjusts the mapping
- Page table is the table that stores the mapping of virtual pages to physical frames

Overview of Paging



VM Address Translation

- Parameters
  - $P = 2^p$ = page size (bytes).
  - $N = 2^n$ = Virtual-address limit
  - $M = 2^m$ = Physical-address limit



Page offset bits don't change as a result of translation

A Background on TLB

- To leverage memory access locality in VA to PA translation, processors with page-based virtual memory use TLB
- TLB is a cache for page table
- TLB caches VA to PA translations

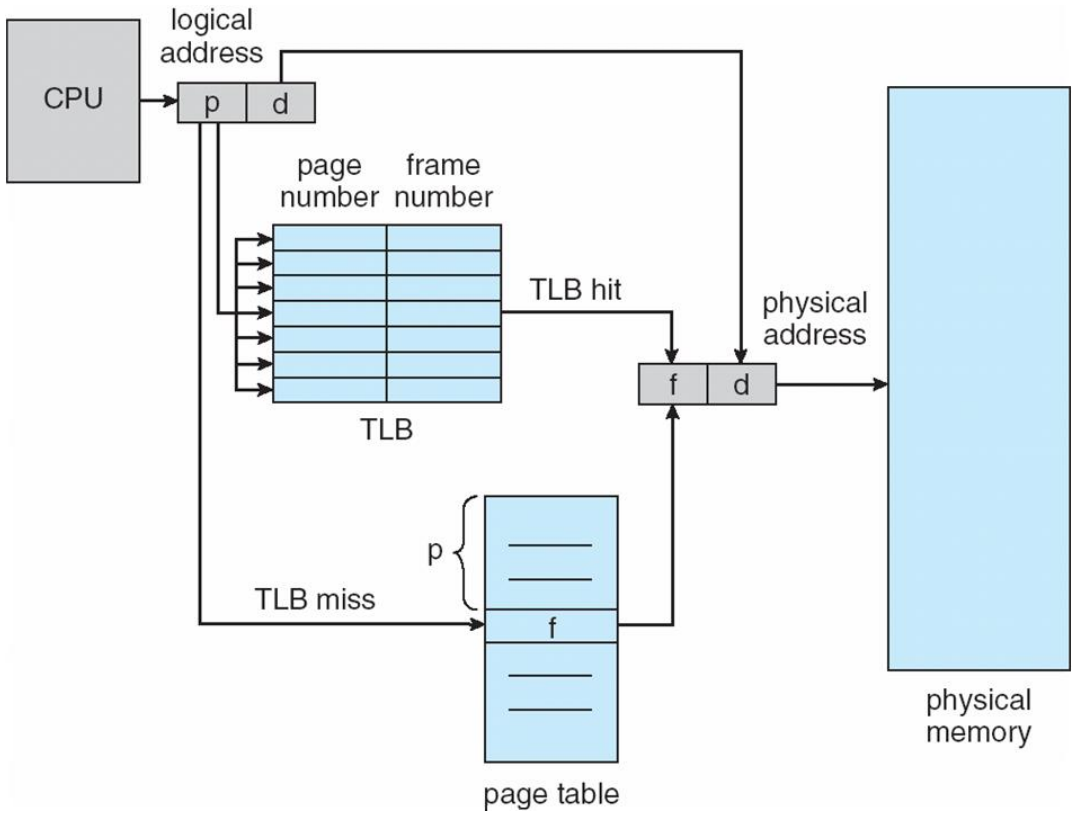


Figure credit: Silberschatz, Galvin and Gagne

TLB types

- We can have separate TLBs for instruction and data
- DTLB and ITLB may be unified or separate
- DTLB miss-rate >> ITLB miss-rate

- TLB can be single or multi-level, similar to cache.
- First level = L1 TLB, second level = L2 TLB

TLB coverage

- **TLB Coverage** is sum of memory mapped by all TLB entries
- E.g., page size = 4KB, # entries = 1024 ➔ coverage = 1024*4KB

Understanding the difference between TLB size and coverage

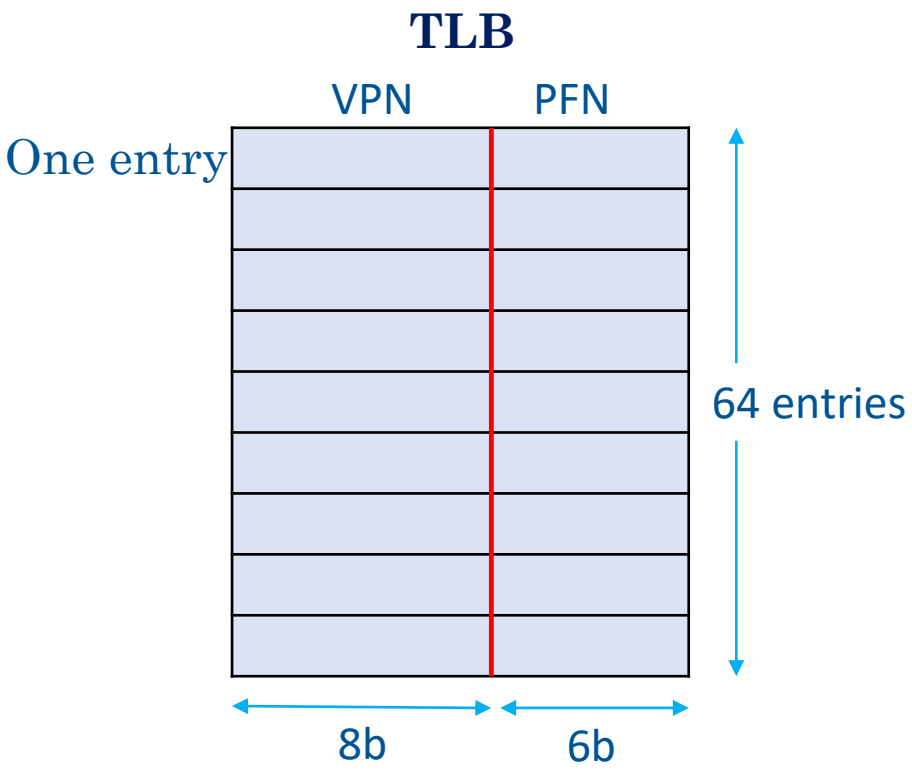
- Question: Let virtual page number be 8b.
- Physical frame number is 6b
- Page-size is 8B.
- TLB has 64 entries

Find TLB size (assuming no valid bit is used) and TLB coverage.

- Answer:
- Size of one entry = 8b+6b = 14b

➔ Size of TLB = 14b*64

- Coverage of TLB = 64*8B



Context Switches

- Assume that Process **X** is running
  - Process **X**'s VPN 5 is mapped to PFN <u>100</u>
  - The TLB caches this mapping
    - VPN 5 → PFN 100

VPN	PFN
5	**100**

- Now assume a context switch to Process **Y**
  - Process **Y**'s VPN 5 is mapped to PFN <u>200 in page table</u>
  - When Process Y tries to access VPN 5, it searches the TLB
    - Process **Y** finds an entry whose VPN value is 5
    - Great! It's a TLB hit!
    - As per TLB, the PFN is found to be 100; but correct PFN is 200 (as per the page table)! TLB is storing wrong (outdated) information.
    - Thus, the hit observed is incorrect/misleading.

Context Switches (Approach 1)

- Approach #1. Flush the TLB
  - On a context switch, invalidate all TLB entries
  - Example: 80836

Original TLB

VPN	PFN
70	800
40	200
50	100
90	400
150	600

After TLB Flush

VPN	PFN

Context Switches (Approach 2)

- Approach #2. Associate TLB entries with the processes
  - All TLB entries have an extra field
    - That identifies the process to which it belongs
  - Invalidate only the entries belonging to the old process
  - Example: Modern x86, MIPS

A TLB With process ID (pID)

pID	VPN	PFN
0	70	800
1	40	200
1	50	100
0	90	400
1	150	600

When process 1 is switched out

pID	VPN	PFN
0	70	800
0	90	400